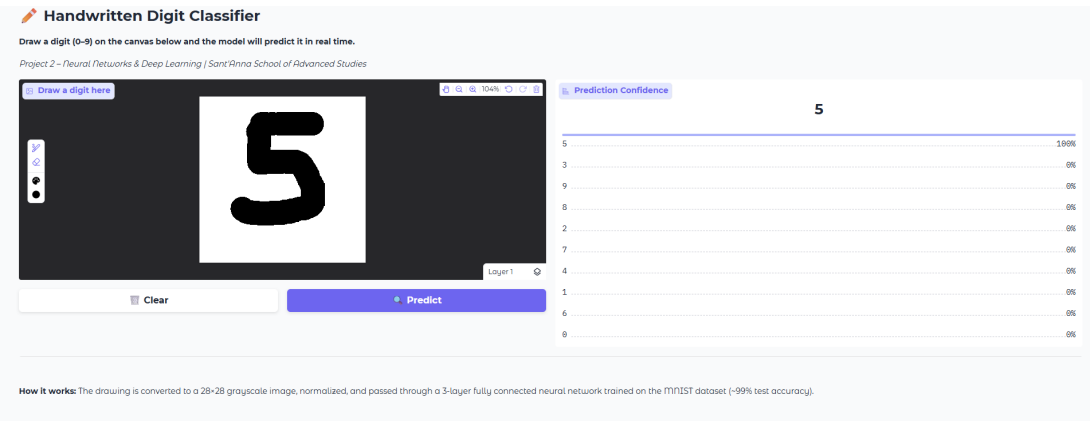
*University of Pisa, Pisa, Italy*
*Sant'Anna School of Advanced Studies, Pisa, Italy*

**Neural Networks & Deep Learning Theoretical Foundations**

# Handwritten Digit Classifier

A Real-Time 3-Layer Neural Network on MNIST




**Author:** Kabir Bakhshaei

**Student ID:** 1312090810

**Email:** kabir.bakhshaei@santannapisa.it

**Program:** Ph.D. in Artificial Intelligence

**Institutions:** Sant'Anna School of Advanced Studies & University of Pisa

**Lab:** SMARTLab, BioRobotics Institute of Sant'Anna

**Supervisor:** Prof. Giovanni Stabile

**Course Professor:** Prof. Giorgio Buttazzo

**Delivery Date:** 2026-04-01

# Contents


**1 Introduction and Application Description** **2**

**2 Neural Network Architecture** **2**
- 2.1 Layer Description . . . 2
- 2.2 Forward Pass . . . 2
- 2.3 Activation Functions . . . 3
- 2.4 Regularization . . . 5
- 2.5 Weight Initialisation . . . 6
- 2.6 Early Stopping . . . 7
- 2.7 Loss Function . . . 7

**3 Implementation Details** **8**
- 3.1 Hardware and Software Environment . . . 8
- 3.2 Programming Language and Framework . . . 9
- 3.3 Dataset . . . 9

**4 Dataset Augmentation** **10**

**5 Hyperparameters** **11**
- 5.1 Training Algorithm . . . 13

**6 Evaluation Metrics** **14**

**7 Experiments and Results** **15**
- 7.1 Training Curves . . . 15
- 7.2 Test Set Performance . . . 17
- 7.3 Real-Time Inference . . . 19

**8 Graphical Interface** **19**
- 8.1 Inference Pipeline . . . 20

**9 Conclusion** **22**

**Appendix** **22**
- A Xavier Initialisation: Worked Example . . . 22
- B Optimiser Comparison: Gradient Descent vs. Adam vs. SOAP . . . 23
- C Epochs and Mini-Batches Explained . . . 24

**References** **26**

## Introduction and Application Description

Handwritten digit recognition is one of the most fundamental problems in machine learning and computer vision. The goal of this project is to design, train, and deploy a fully connected neural network that classifies handwritten digits (0 through 9) drawn by a user in real time via a graphical interface.

The application is built around the **MNIST dataset** [1], which stands for Modified National Institute of Standards and Technology, one of the most widely used benchmarks in pattern recognition. The system takes a 28×28 grayscale image drawn by the user as input and produces a probability distribution over 10 digit classes at the output. The user interacts with the model through a browser-based drawing canvas, and the predicted digit is shown instantly alongside the confidence score for each class.

This type of application is directly relevant to real-world systems such as postal code recognition, bank cheque processing, and form digitization. It also serves as a concrete illustration of key concepts in deep learning: feedforward architectures, supervised training with backpropagation, regularization, and softmax classification.

## Neural Network Architecture

The network follows the 3-layer fully connected architecture specified in the project description. Given an input image of size $28 \times 28 = 784$ pixels, the network maps it to a 10-dimensional output vector representing class probabilities.

### Layer Description

Table 1 summarises the layer-by-layer structure of the network, including the input and output dimensions and the activation function applied at each stage.

| Layer | Input Size | Output Size | Activation |
|---|---|---|---|
| Flatten | $28 \times 28$ | 784 | — |
| FC1 | 784 | 512 | ReLU |
| Dropout | — | — | $p = 0.3$ |
| FC2 | 512 | 256 | ReLU |
| Dropout | — | — | $p = 0.3$ |
| FC3 (Output) | 256 | 10 | Softmax |

**Table 1:** Layer-by-layer description of the DigitClassifier network.

### Forward Pass

Let $\mathbf{x} \in \mathbb{R}^{784}$ denote the flattened input image. The forward pass through the network computes:

$$\mathbf{h}^{(1)} = \mathrm{ReLU}\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right), \quad \mathbf{W}^{(1)} \in \mathbb{R}^{512\times 784} \tag{1}$$

$$\mathbf{h}^{(2)} = \mathrm{ReLU}\left(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\right), \quad \mathbf{W}^{(2)} \in \mathbb{R}^{256\times 512} \tag{2}$$

$$\mathbf{z} = \mathbf{W}^{(3)}\mathbf{h}^{(2)} + \mathbf{b}^{(3)}, \quad \mathbf{W}^{(3)} \in \mathbb{R}^{10\times 256} \tag{3}$$

$$\hat{\mathbf{p}} = \mathrm{Softmax}(\mathbf{z}) \in \mathbb{R}^{10} \tag{4}$$

where $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the learnable weight matrix and bias vector of layer $l$, and Dropout is applied to $\mathbf{h}^{(1)}$ and $\mathbf{h}^{(2)}$ during training only. The exact total number of trainable parameters is:

$$(512\times 784+512)+(256\times 512+256)+(10\times 256+10) = 401{,}920+131{,}328+2{,}570 = 535{,}818 \tag{5}$$

Figure 1 illustrates the full architecture alongside the loss computation and backpropagation loop, while Figure 9 shows the end-to-end inference pipeline from user input to final prediction.

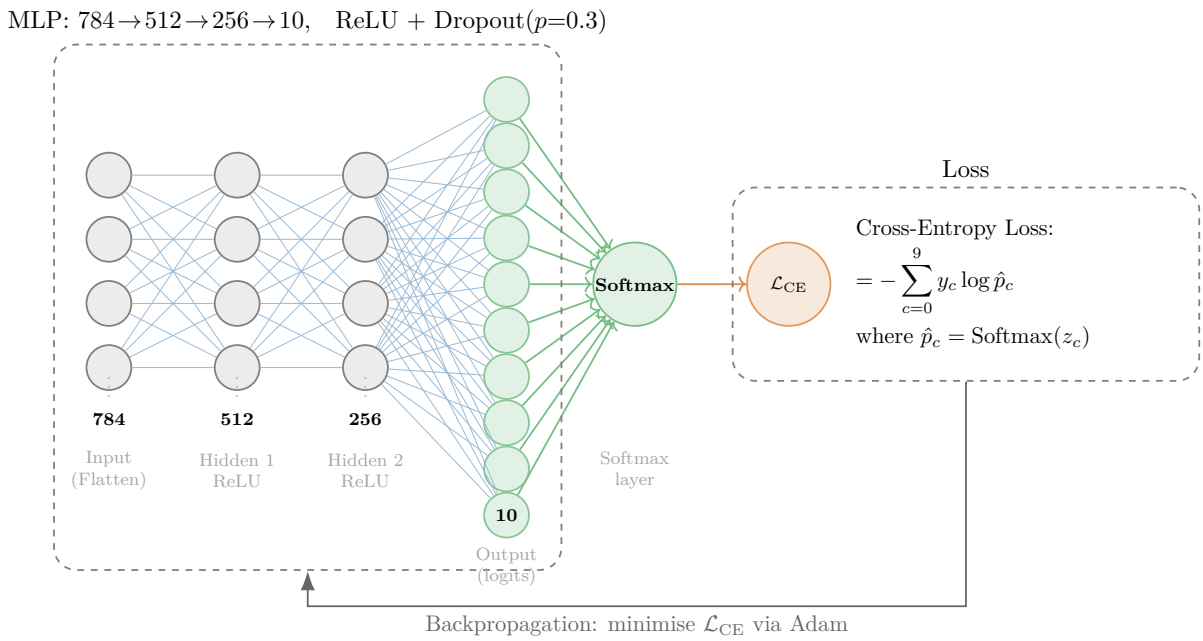



**Figure 1:** Neural network architecture and loss function for the Digit Classifier. The network maps a flattened 28×28 image (784 inputs) through two fully connected hidden layers (512 and 256 neurons, ReLU activation, Dropout $p$=0.3) to a 10-class softmax output. Training minimises the Cross-Entropy loss via the Adam optimiser.

The following subsections describe in detail each of the key design decisions reflected in the architecture diagram and flowchart above: the choice of activation function, weight initialisation strategy, regularisation technique, early stopping criterion, and loss function.

### Activation Functions

The **ReLU** (Rectified Linear Unit) activation function is used after each hidden layer. ReLU is defined as $f(x) = \max(0, x)$ and is preferred over sigmoid or tanh in deep networks

because it avoids the vanishing gradient problem and converges faster in practice [5].

Figure 2 compares the three most common activation functions used in deep learning. ReLU is chosen in this project because it does not saturate for positive values, keeping gradients alive during backpropagation.

Although ReLU significantly alleviates the vanishing gradient problem, it is important to note that it is not entirely free from it. For negative inputs ($x < 0$), the ReLU function outputs zero and its gradient is also zero. As a result, neurons receiving consistently negative inputs may stop updating their weights during training, a phenomenon commonly referred to as the *dying ReLU* problem.

In contrast, sigmoid and tanh activation functions suffer from a more severe form of the vanishing gradient problem due to their *saturation regions*. For large positive or negative input values, these functions approach constant output values (e.g., 0 or 1 for sigmoid, and $-1$ or 1 for tanh), where their gradients become very small (close to zero). This causes gradients to diminish as they propagate backward through deep networks, significantly slowing down or even preventing effective learning.

ReLU, however, does not saturate for positive inputs ($x > 0$), where its gradient remains constant and equal to 1. This property allows gradients to flow efficiently during backpropagation, making ReLU a preferred choice in deep neural networks despite its limitations on the negative side.

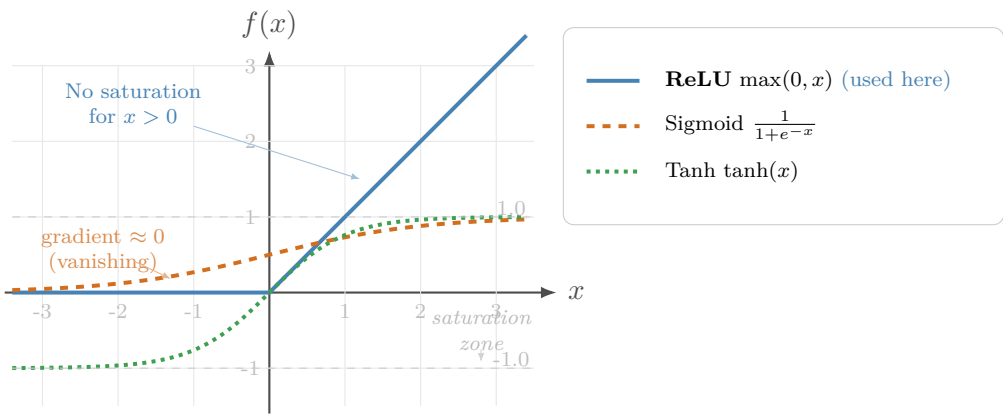



**Figure 2:** Comparison of activation functions. **ReLU** (solid blue) is used after each hidden layer in this project because its gradient is exactly 1 for all $x > 0$, preventing the vanishing gradient problem. Sigmoid (dashed orange) and Tanh (dotted green) both saturate for large $|x|$, causing gradients to approach zero and slowing down training in deep networks [5].

The **Softmax** function is applied at the output to convert raw logits into a valid probability distribution over the 10 digit classes:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=0}^{9} e^{z_j}} \tag{6}$$

**Worked Example.** Suppose the network receives an image of the digit **3** and produces the following raw logits from the output layer (one value per digit class):

$$\mathbf{z} = [-1.2,\ 0.5,\ 0.3,\ \mathbf{2.8},\ -0.4,\ 0.1,\ -0.9,\ 0.2,\ -0.1,\ 0.0]$$

Applying Softmax, the probability for digit 3 (the correct class) is:

$$\hat{p}_3 = \frac{e^{2.8}}{e^{-1.2} + e^{0.5} + e^{0.3} + e^{2.8} + e^{-0.4} + e^{0.1} + e^{-0.9} + e^{0.2} + e^{-0.1} + e^{0.0}} = \frac{16.44}{21.97} \approx 0.748 \tag{7}$$

So the model assigns a **74.8% confidence** to digit 3, while the remaining 25.2% is distributed across the other 9 classes. The predicted digit is then:

$$\hat{c} = \arg\max_c \ \hat{p}_c = 3 \tag{8}$$

which correctly identifies the input as digit 3. Note that all 10 output probabilities sum to exactly 1, making $\hat{\mathbf{p}}$ a valid probability distribution.

### Regularization

**Dropout** with a probability of $p = 0.3$ is applied after each hidden layer during training. Dropout randomly deactivates 30% of neurons per forward pass, which prevents the network from relying too heavily on specific neurons and reduces overfitting. During inference, dropout is disabled and all neurons are active [3].

Figure 3 illustrates how Dropout is applied across the two hidden layers of the DigitClassifier during training versus inference.

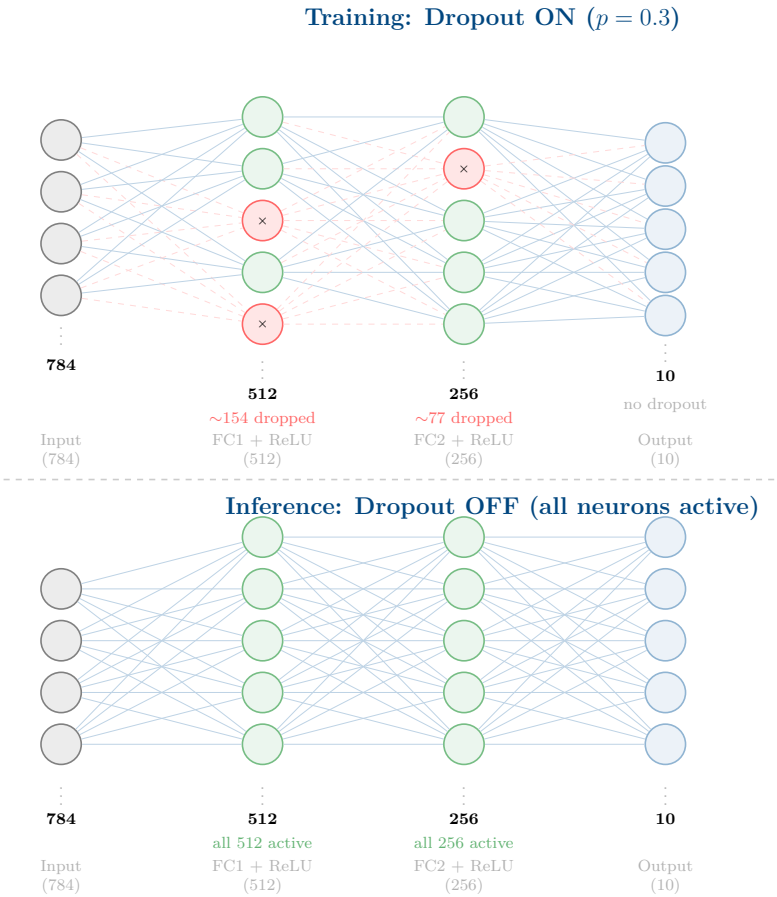



**Figure 3:** Dropout applied to the DigitClassifier architecture. *Top:* During training, approximately 30% of neurons in FC1 (∼154 of 512) and FC2 (∼77 of 256) are randomly deactivated per forward pass (shown in red with ×), with all incoming and outgoing connections disabled (dashed lines). The output layer is never dropped. *Bottom:* During inference, Dropout is disabled and all 512 neurons in FC1 and all 256 neurons in FC2 are fully active, ensuring deterministic and stable predictions.

### Weight Initialisation

Network weights are initialised using **Xavier uniform initialisation** [4], also known as Glorot initialisation. For each Linear layer, weights are drawn from a uniform distribution:

$$W \sim \mathcal{U}(-a,\, a)\,, \qquad a = \sqrt{\frac{6}{\text{fan_in} + \text{fan_out}}} \tag{9}$$

where fan_in and fan_out are the number of input and output neurons of the layer respectively. All biases are initialised to zero.

Xavier initialisation is preferred over random normal or uniform initialisation because it keeps the variance of activations approximately constant across layers, preventing the gradients from vanishing or exploding at the start of training. This is particularly important for deep networks with ReLU activations, where poorly scaled initial weights can cause neurons to be permanently inactive from the first forward pass.

Table 2 shows the exact Xavier initialisation bounds computed for each layer of the

DigitClassifier, using the actual fan_in and fan_out values of the network.

| **Layer** | **fan_in** | **fan_out** | $a = \sqrt{6/(\textbf{fan_in} + \textbf{fan_out})}$ | **Weight Range** |
|---|---|---|---|---|
| FC1 | 784 | 512 | $\sqrt{6/1296} = 0.0680$ | $[-0.0680,\ +0.0680]$ |
| FC2 | 512 | 256 | $\sqrt{6/768} = 0.0884$ | $[-0.0884,\ +0.0884]$ |
| FC3 | 256 | 10 | $\sqrt{6/266} = 0.1501$ | $[-0.1501,\ +0.1501]$ |

**Table 2:** Xavier uniform initialisation bounds for each layer of the DigitClassifier. Note that $a$ increases toward the output layer as the number of neurons decreases, maintaining a consistent activation variance throughout the network.

### Early Stopping

To avoid wasting computational resources and prevent overfitting in later epochs, **early stopping** is applied during training. The validation loss is monitored after each epoch, and training is automatically halted if for 7 consecutive epochs the score has not improved by at least $\delta = 0.0001$ compared to the best score ever.

$$\mathcal{L}_{\text{val}}^{(t)} \geq \mathcal{L}_{\text{val}}^{*} - \delta \quad \text{for } t = e, e+1, \ldots, e+p \tag{10}$$

where $\mathcal{L}_{\text{val}}^{*}$ is the best validation loss observed so far, $p = 7$ is the patience, and $\delta = 0.0001$ is the minimum improvement threshold. Whenever a new best is found, the model weights are saved immediately, so the final saved model always corresponds to the epoch with the lowest validation loss, not the last epoch.

### Loss Function

The network is trained by minimising the **Cross-Entropy Loss** $\mathcal{L}_{\text{CE}}$, which measures the dissimilarity between the predicted probability distribution $\hat{\mathbf{p}}$ and the true one-hot encoded label vector $\mathbf{y}$:

$$\mathcal{L}_{\text{CE}} = -\sum_{c=0}^{9} y_c \log \hat{p}_c \tag{11}$$

where $y_c \in \{0, 1\}$ is 1 only for the true class $c$, and $\hat{p}_c$ is the predicted probability for class $c$ obtained from the Softmax output:

$$\hat{p}_c = \frac{e^{z_c}}{\sum_{j=0}^{9} e^{z_j}} \tag{12}$$

Since only one class is correct per sample, the loss simplifies to:

$$\mathcal{L}_{\text{CE}} = -\log \hat{p}_{c^*} \tag{13}$$

where $c^*$ is the index of the ground-truth digit. Minimising this loss encourages the network to assign maximum probability mass to the correct class. In PyTorch, `nn.CrossEntropyLoss` combines the Softmax and the cross-entropy computation in a single numerically stable operation, so the output layer produces raw logits $z_c$ rather than probabilities.

**Worked Example.** Suppose the network receives an image of digit **3**. The true label is represented as a one-hot vector $\mathbf{y}$:

$$\mathbf{y} = [\underbrace{0}_{0}, \underbrace{0}_{1}, \underbrace{0}_{2}, \underbrace{1}_{3}, \underbrace{0}_{4}, \underbrace{0}_{5}, \underbrace{0}_{6}, \underbrace{0}_{7}, \underbrace{0}_{8}, \underbrace{0}_{9}]$$

Only $y_3 = 1$, all others are zero. The network produces the following Softmax probabilities $\hat{\mathbf{p}}$:

$$\hat{\mathbf{p}} = [0.01,\ 0.02,\ 0.03,\ \mathbf{0.75},\ 0.04,\ 0.03,\ 0.02,\ 0.05,\ 0.03,\ 0.02]$$

Applying the cross-entropy formula:

$$\begin{aligned}
\mathcal{L}_{\text{CE}} &= -\sum_{c=0}^{9} y_c \log \hat{p}_c \\
&= -(y_0 \log \hat{p}_0 + y_1 \log \hat{p}_1 + \cdots + y_3 \log \hat{p}_3 + \cdots + y_9 \log \hat{p}_9) \\
&= -(0 \cdot \log 0.01 + 0 \cdot \log 0.02 + \cdots + \mathbf{1} \cdot \log \mathbf{0.75} + \cdots + 0 \cdot \log 0.02) \\
&= -\log(0.75) = 0.288
\end{aligned} \tag{14}$$

Since $y_c = 0$ for all classes except $c^* = 3$, every term vanishes and only $-\log \hat{p}_3$ survives. Now consider two extreme cases to build intuition:

| **Scenario** | $\hat{p}_3$ **(correct class)** | $\mathcal{L}_{\text{CE}} = -\log \hat{p}_3$ | **Interpretation** |
|---|---|---|---|
| Perfect prediction | 1.00 | $-\log(1.00) = 0.000$ | No loss, perfect |
| Good prediction | 0.75 | $-\log(0.75) = 0.288$ | Small loss |
| Poor prediction | 0.10 | $-\log(0.10) = 2.303$ | Large loss |
| Wrong prediction | 0.01 | $-\log(0.01) = 4.605$ | Very large loss |

**Table 3:** Cross-entropy loss as a function of the predicted probability for the correct class (digit 3). The loss grows rapidly as confidence in the correct class decreases, strongly penalising wrong predictions.

This shows why minimising $\mathcal{L}_{\text{CE}}$ during training directly pushes the network to assign higher probability to the correct digit, the only way to reduce the loss is to increase $\hat{p}_{c^*}$.

## Implementation Details

### Hardware and Software Environment

All training and evaluation experiments were conducted on a personal laptop. Table 4 summarises the hardware and software configuration used.

| Component | Specification |
|---|---|
| CPU | Intel Core i7-12650H |
| GPU | NVIDIA RTX 3060 Laptop (6 GB VRAM) |
| RAM | 16 GB |
| NVIDIA Driver | 566.07 |
| CUDA Toolkit | 12.1 |
| Operating System | Windows 11 |
| Conda Environment | `pinns_cuda` |
| Python | 3.10.19 |
| PyTorch | 2.5.1 (CUDA 12.1, cuDNN 9) |
| TorchVision | 0.20.1 |
| NumPy | 2.0.1 |
| Matplotlib | 3.10.8 |
| Pillow | 11.1.0 |
| PyYAML | 6.0.3 |
| Gradio | 6.10.0 |
| Seaborn | 0.13.2 |
| Scikit-learn | 1.7.2 |

**Table 4:** Hardware and software environment used for training and evaluation.

The NVIDIA RTX 3060 GPU was used to accelerate training via CUDA 12.1. PyTorch automatically detects and utilises the GPU at runtime through `torch.cuda.is_available()`, reducing per-epoch training time significantly compared to CPU-only execution. Inference in the Gradio application runs on CPU in real time, with each forward pass completing in under 1 ms.

### Programming Language and Framework

The project is implemented in **Python 3.10.19** using **PyTorch 2.5.1** as the deep learning framework. The graphical user interface is built with **Gradio 6.10.0**, which provides a browser-based drawing canvas with real-time model inference.

The codebase is organised following a modular MLOps-style structure, with all hyperparameters centralised in `config.yaml`:

- `src/model.py`: network architecture and Xavier initialisation
- `src/train.py`: training loop, early stopping, LR scheduling
- `src/evaluate.py`: test set evaluation and metric reporting
- `src/utils.py`: helper functions (preprocessing, save/load)
- `app.py`: Gradio web interface for real-time inference
- `config.yaml`: all hyperparameters and paths in one place

### Dataset

The **MNIST dataset** consists of 70,000 grayscale images of handwritten digits: 60,000 for training and 10,000 for testing. Each image is $28 \times 28$ pixels, with pixel values in $[0, 255]$. The dataset is balanced across 10 classes (digits 0–9), with approximately 6,000 samples per class in the training set.

A **validation split** of 10% was carved out from the training set, resulting in:

- Training set: 54,000 images
- Validation set: 6,000 images
- Test set: 10,000 images

Input images are normalized using the MNIST dataset mean ($\mu = 0.1307$) and standard deviation ($\sigma = 0.3081$):

$$\hat{x} = \frac{x - \mu}{\sigma} \tag{15}$$

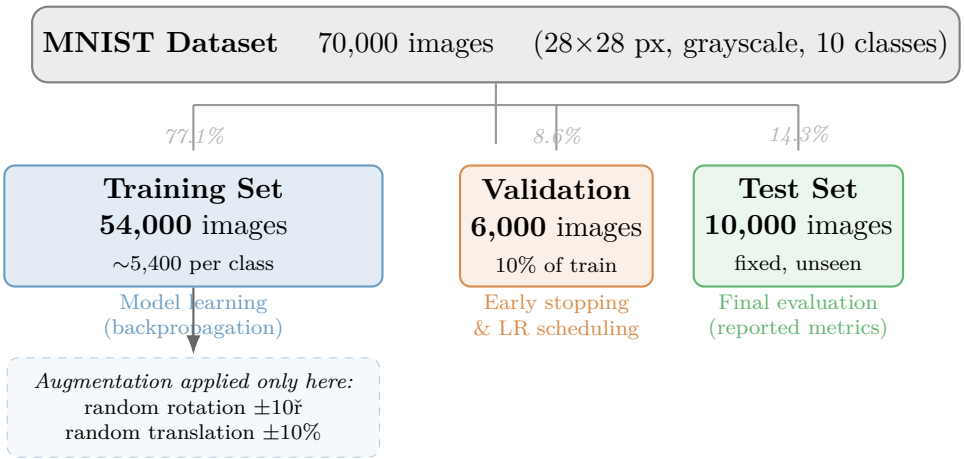



**Figure 4:** MNIST dataset split used in this project. The original 60,000 training images are further divided into 54,000 training and 6,000 validation images. Data augmentation (random rotation and translation) is applied exclusively to the training set. The test set of 10,000 images remains untouched throughout training and is used only for the final evaluation.

## Dataset Augmentation

To improve the model's robustness to different handwriting styles and to reduce overfitting, the following data augmentation techniques are applied *only* during training:

- **Random Rotation:** images are randomly rotated by up to ±10 degrees, simulating slight tilts in real handwriting.
- **Random Affine Translation:** images are randomly shifted horizontally and vertically by up to 10% of the image size, to account for off-center drawings.

No augmentation is applied to the validation or test sets, ensuring that evaluation metrics reflect clean, unmodified inputs.

| Split | Samples | Classes | Image Size |
|---|---|---|---|
| Training | 54,000 | 10 (digits 0–9) | 28 × 28 px |
| Validation | 6,000 | 10 (digits 0–9) | 28 × 28 px |
| Test | 10,000 | 10 (digits 0–9) | 28 × 28 px |
| **Total** | **70,000** | — | — |

**Table 5:** MNIST dataset split used for training, validation, and testing.

## Hyperparameters

All hyperparameters were selected based on standard practice for MNIST classification and verified through validation performance during training.

| **Hyperparameter** | **Value** |
|---|---|
| Batch size | 64 |
| Learning rate (initial) | 0.001 |
| Max epochs | 50 (early stopping active) |
| Optimizer | Adam |
| Loss function | Cross-Entropy |
| Dropout probability | 0.3 |
| LR scheduler | ReduceLROnPlateau (patience=3, factor=0.5, min_lr=$10^{-5}$) |
| Early stopping patience | 7 epochs |
| Early stopping $\delta$ | 0.0001 |
| Validation split | 10% |

**Table 6:** Hyperparameter configuration used for training.

The **Adam optimizer** [2] was chosen because it combines the benefits of momentum and adaptive learning rates, making it well-suited for training deep networks. The **ReduceLROnPlateau** scheduler reduces the learning rate by a factor of 0.5 whenever the validation loss shows no improvement for 3 consecutive epochs, down to a minimum of $10^{-5}$, helping the model converge to a better minimum in later training stages.

Before presenting the update equations, it is helpful to understand the two memory variables that Adam maintains for each weight:

- $\mathbf{m}_t$: **momentum**: a running average of recent gradients, tracking *which direction* the weights have been moving. If gradients have been consistently positive for several steps, $\mathbf{m}_t$ will be positive and Adam will continue moving in that direction even if the current gradient is noisy.
- $\mathbf{v}_t$: **velocity**: a running average of recent *squared* gradients, tracking *how steep* the loss surface has been. A large $\mathbf{v}_t$ means the gradient has been large recently, so Adam takes a smaller step to avoid overshooting. A small $\mathbf{v}_t$ means the surface is flat, so Adam takes a larger step to avoid stalling.

Both variables are initialised to zero before training begins: $\mathbf{m}_0 = \mathbf{0}$, $\mathbf{v}_0 = \mathbf{0}$. At each step $t$, they are updated using the current gradient $\mathbf{g}_t = \nabla_\theta \mathcal{L}_t$, the vector of partial derivatives of the loss with respect to all weights, computed via backpropagation. Intuitively, $\mathbf{g}_t$ answers the question: *"for each weight, which direction reduces the loss?"*

Formally, given the gradient $\mathbf{g}_t = \nabla_\theta \mathcal{L}_t$ at step $t$, Adam updates the parameters as follows:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t \tag{16}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2 \tag{17}$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t} \tag{18}$$

$$\theta_t = \theta_{t-1} - \eta \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \tag{19}$$

where $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ are the default PyTorch values, and $\eta = 0.001$ is the initial learning rate used in this project.

All the above components, Xavier initialisation, Adam optimisation, learning rate scheduling, and early stopping, are integrated into a single training loop summarised in Algorithm 1 below.

**Worked Example: One Weight, Three Steps.** To make the Adam update concrete, consider a single weight $w$ in the network tracked over three training steps, with the following setup:

$$\eta = 0.001, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-8}, \quad w_0 = 0.5, \quad m_0 = 0, \quad v_0 = 0$$

Suppose backpropagation returns gradients $g_1 = 0.3$, $g_2 = 0.2$, $g_3 = 0.4$ at steps 1, 2, and 3 respectively. Table 7 traces the full Adam computation at each step.

| **Step** $t$ | $g_t$ | $m_t$ | $v_t$ | $\hat{m}_t$ | $\hat{v}_t$ | $w_t$ |
|---|---|---|---|---|---|---|
| 0 | — | 0.0000 | 0.000000 | — | — | 0.5000 |
| 1 | 0.3 | 0.0300 | 0.000090 | 0.300 | 0.090 | 0.4990 |
| 2 | 0.2 | 0.0470 | 0.000130 | 0.247 | 0.065 | 0.4980 |
| 3 | 0.4 | 0.0823 | 0.000290 | 0.304 | 0.097 | 0.4970 |

**Table 7:** Adam update trace for a single weight $w$ over three training steps. $m_t$ accumulates gradient direction (momentum); $v_t$ accumulates gradient magnitude (velocity). Both are bias-corrected into $\hat{m}_t$ and $\hat{v}_t$ before the weight update.

At step $t = 1$, the full computation is:

$$m_1 = 0.9 \times 0 + 0.1 \times 0.3 = 0.030 \tag{20}$$

$$v_1 = 0.999 \times 0 + 0.001 \times (0.3)^2 = 0.000090 \tag{21}$$

$$\hat{m}_1 = \frac{0.030}{1-0.9^1} = \frac{0.030}{0.1} = 0.300 \tag{22}$$

$$\hat{v}_1 = \frac{0.000090}{1-0.999^1} = \frac{0.000090}{0.001} = 0.090 \tag{23}$$

$$w_1 = 0.500 - 0.001 \times \frac{0.300}{\sqrt{0.090} + 10^{-8}} = 0.500 - 0.001 \times 1.000 = \mathbf{0.499} \tag{24}$$

Note that $\hat{m}_1/\sqrt{\hat{v}_1} = 0.300/0.300 = 1.000$ at step 1: Adam naturally normalises the update to approximately 1 at the start, so the effective step size is simply $\eta = 0.001$ regardless of the gradient magnitude. This is the key advantage over plain gradient descent, which would take a step of $\eta \cdot g_1 = 0.001 \times 0.3 = 0.0003$, three times smaller.

**Why bias correction matters.** Early in training, $m_t$ and $v_t$ are initialised to zero and are therefore underestimated. Without bias correction ($\hat{m}_t$, $\hat{v}_t$), the first few updates would be artificially small and training would start too slowly. The correction terms $1-\beta_1^t$ and $1-\beta_2^t$ grow from near-zero toward 1 as $t$ increases, gradually removing this bias after the first 10 steps.

**In my training.** Adam performed this exact computation for all **535,818 weights simultaneously**, at every mini-batch update, approximately $50 \times 844 = 42{,}200$ times in total across the full 50 epochs, each time adjusting every weight by its own individually adapted step size.

### Training Algorithm

Algorithm 1 summarises the full training procedure including early stopping and learning rate scheduling.

**Algorithm 1** Training with Early Stopping and LR Scheduling

**Require:** Dataset $\mathcal{D}$, config (lr= 0.001, batch_size= 64, patience $p = 7$, $\delta = 0.0001$, $E_{\max} = 50$)
**Ensure:** Trained model weights $\theta^*$

```
Initialise model weights θ with Xavier uniform [4]
Initialise Adam optimiser with η = 0.001, β1 = 0.9, β2 = 0.999
L* ← ∞,   counter ← 0
for epoch = 1, 2, ..., E_max do
    for each mini-batch (x_b, y_b) ∈ D_train do
        p̂ ← Softmax(f_θ(x_b))
        L_CE ← − Σ_c y_c log p̂_c
        θ ← θ − η · ∇_θ L_CE                          (Adam update)
    end for
    Compute L_val on validation set
    Update η via ReduceLROnPlateau(L_val)
    if L_val < L* − δ then
        L* ← L_val,   counter ← 0,   θ* ← θ            (save best model)
    else
        counter ← counter +1
        if counter ≥ p then
            break                                      (early stopping)
        end if
    end if
end for
return θ*
```

## Evaluation Metrics

The following metrics are computed on the held-out test set of 10,000 samples using the `scikit-learn` library. Let $\mathrm{TP}_c$, $\mathrm{FP}_c$, $\mathrm{FN}_c$ denote the true positives, false positives, and false negatives for class $c$, respectively.

Before defining each metric formally, it is helpful to understand the four possible outcomes when the model classifies a single image for a given class $c$. Consider digit 3 as an example:

| | **Model predicts: 3** | **Model predicts: not 3** |
|---|---|---|
| **Image is: 3** | **True Positive (TP)**: correct | **False Negative (FN)**: missed |
| **Image is: not 3** | **False Positive (FP)**: false alarm | **True Negative (TN)**: correct |

**Table 8:** The four possible classification outcomes for a given class $c$. TP and TN are correct decisions; FP and FN are errors.

Intuitively: **True/False** indicates whether the model was correct, and **Positive/Negative** indicates what the model predicted. A **False Positive** (FP) occurs when the model incorrectly calls an image "3" when it is actually another digit. A **False Negative** (FN) occurs when the model fails to recognise a real 3 and assigns it a different label. These four quantities form the foundation of all evaluation metrics defined below.

- **Accuracy:** the fraction of correctly classified samples over all test samples:

$$\text{Accuracy} = \frac{\sum_{c=0}^{9} \mathrm{TP}_c}{N} \tag{25}$$

  where $N = 10{,}000$ is the total number of test samples.

- **Precision** (per class): the proportion of predicted positives that are truly positive:

$$\text{Precision}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c} \tag{26}$$

- **Recall** (per class): the proportion of actual positives correctly identified:

$$\text{Recall}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c} \tag{27}$$

- **F1 Score** (per class): the harmonic mean of precision and recall, balancing both measures:

$$\mathrm{F1}_c = 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \tag{28}$$

- **Confusion Matrix:** a $10 \times 10$ matrix $C$ where entry $C_{ij}$ counts the number of samples of true class $i$ predicted as class $j$. A perfect classifier produces a diagonal matrix.

| | **Predicted: 3** | **Predicted: not 3** |
|---|---|---|
| **True: 3** | TP = 1003 | FN = 7 |
| **True: not 3** | FP = 10 | TN = 8980 |

**Table 9:** Binary confusion breakdown for digit 3 on the MNIST test set.

**Worked Example: Digit 3.** To make these metrics concrete, consider how they are computed for digit 3 using the actual results from Table 10. Suppose out of 10,000 test samples we focus on the binary question: *"Is this digit a 3 or not?"*

Using these values:

$$\text{Precision}_3 = \frac{\text{TP}_3}{\text{TP}_3 + \text{FP}_3} = \frac{1003}{1003 + 10} = \frac{1003}{1013} = 0.9901 \tag{29}$$

$$\text{Recall}_3 = \frac{\text{TP}_3}{\text{TP}_3 + \text{FN}_3} = \frac{1003}{1003 + 7} = \frac{1003}{1010} = 0.9931 \tag{30}$$

$$\text{F1}_3 = 2 \cdot \frac{0.9901 \times 0.9931}{0.9901 + 0.9931} = 2 \cdot \frac{0.9832}{1.9832} = 0.9916 \tag{31}$$

These values match exactly the results reported in Table 10. Now the intuition behind each metric becomes clear:

- **Precision = 0.9901** means: of all images the model called "3", 99.01% really were 3. Only 10 images from other classes were wrongly labelled as 3 (**false positives**).
- **Recall = 0.9931** means: of all 1,010 real 3s in the test set, the model correctly found 99.31% of them. Only 7 actual 3s were missed (**false negatives**).
- **F1 = 0.9916** is the single balanced score combining both, useful when you care equally about missing digits and mislabelling them.

Finally, the overall accuracy across all 10 classes:

$$\text{Accuracy} = \frac{\sum_{c=0}^{9} \text{TP}_c}{N} = \frac{9{,}898}{10{,}000} = 98.98\% \tag{32}$$

which matches the test accuracy reported in Section 7.

## Experiments and Results

### Training Curves

Training ran for the full 50 epochs without early stopping being triggered, meaning the validation loss continued to show marginal improvement throughout training and never plateaued for 7 consecutive epochs. The best validation loss achieved was 0.0626 at a validation accuracy of 98.4%.

As shown in Figure 5, the training loss dropped sharply in the first 5 epochs (from $\sim$0.60 to $\sim$0.20), then decreased gradually through epoch 50 to a final value of 0.0803. The validation loss remained consistently *below* the training loss throughout, an expected effect of

Dropout, which is active during training but disabled at validation time, making training harder than evaluation. Both curves converge smoothly with no sign of overfitting.

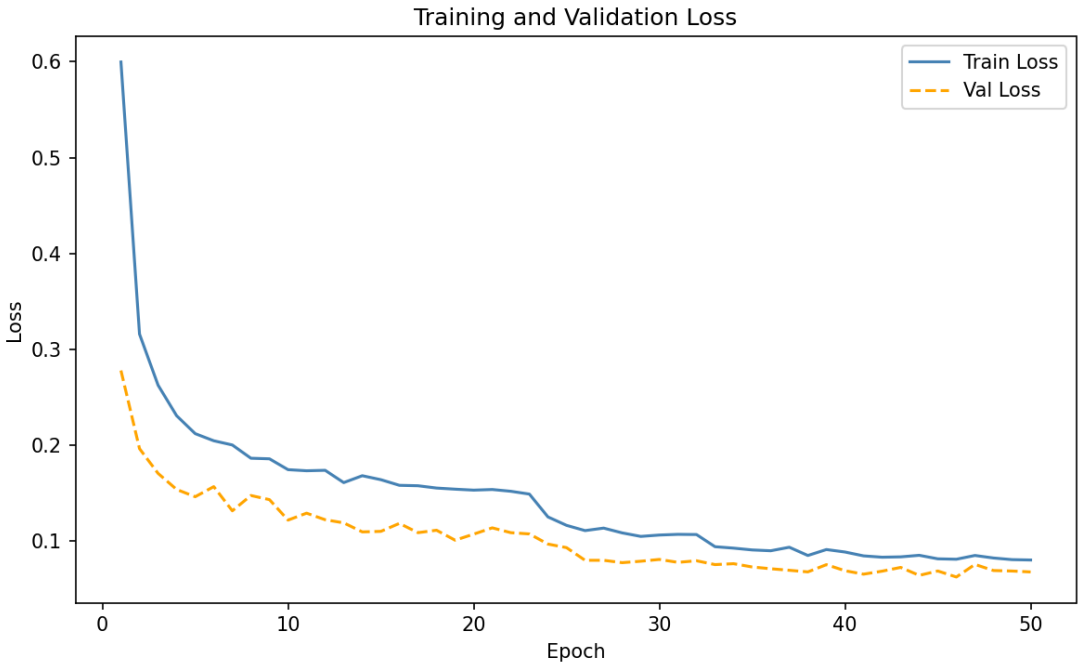



**Figure 5:** Training and validation loss over 50 epochs. The validation loss lies consistently below the training loss due to Dropout being disabled at evaluation time. Both curves decrease smoothly with no sign of overfitting.

Figure 6 shows the corresponding accuracy curves. Training accuracy started at 80.6% in epoch 1 and reached 97.6% by epoch 50, while validation accuracy started higher at 91.4% (again due to Dropout being off at validation) and stabilised around 97.8%–98.4% in the final epochs.

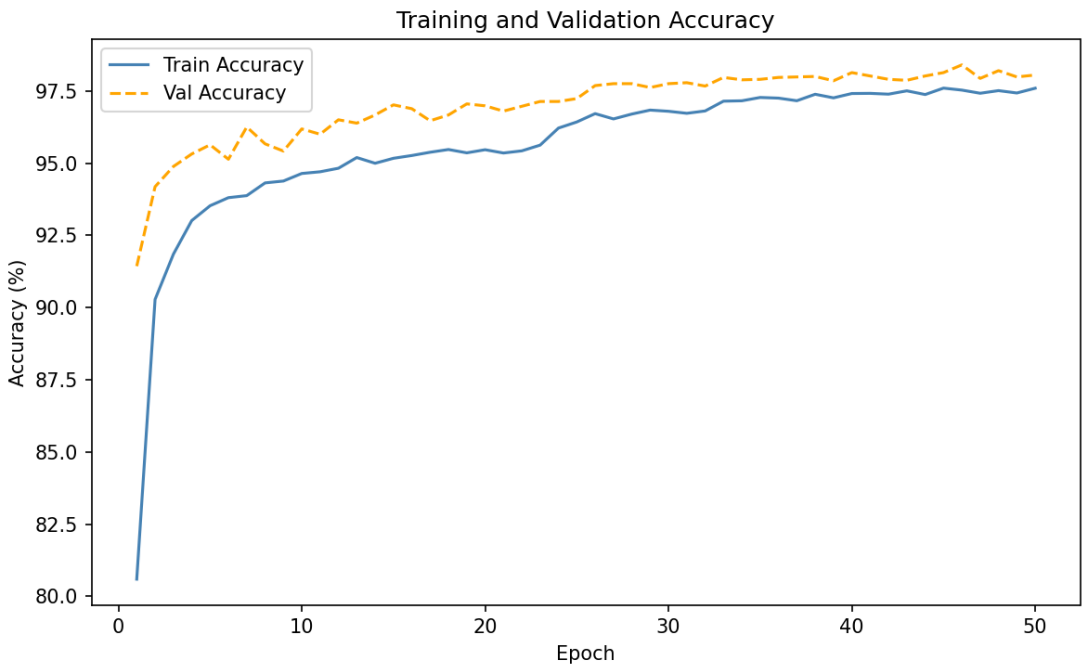



**Figure 6:** Training and validation accuracy over 50 epochs. Validation accuracy consistently exceeds training accuracy for the same reason as above (Dropout). Both converge to approximately 97.5%–98.4% by the final epochs.

### Test Set Performance

As shown in 10, the trained model achieves a **test accuracy of 98.98%** on the full 10,000-sample MNIST test set. The macro-averaged precision, recall, and F1 score are all 0.9898, confirming that performance is consistently high across all 10 digit classes with no class being systematically harder than others. Table 10 reports the exact per-class metrics. The hardest digits to classify are 7 (F1 = 0.9834) and 9 (F1 = 0.9831), which is consistent with the visual similarity between these digits and others such as 1, 4, and 8.

| Digit | Precision | Recall | F1 Score | Support |
|---|---|---|---|---|
| 0 | 0.9909 | 0.9959 | 0.9934 | 980 |
| 1 | 0.9921 | 0.9991 | 0.9956 | 1135 |
| 2 | 0.9904 | 0.9961 | 0.9932 | 1032 |
| 3 | 0.9901 | 0.9931 | 0.9916 | 1010 |
| 4 | 0.9868 | 0.9919 | 0.9893 | 982 |
| 5 | 0.9921 | 0.9865 | 0.9893 | 892 |
| 6 | 0.9916 | 0.9875 | 0.9895 | 958 |
| 7 | 0.9844 | 0.9825 | 0.9834 | 1028 |
| 8 | 0.9928 | 0.9846 | 0.9887 | 974 |
| 9 | 0.9870 | 0.9792 | 0.9831 | 1009 |
| **Macro Avg** | **0.9898** | **0.9896** | **0.9897** | **10000** |

**Table 10:** Per-class classification report on the MNIST test set (10,000 samples).

Figure 7 shows the confusion matrix. The diagonal is strongly dominant, confirming very

few misclassifications. The most common error is digit 9 being confused with digit 4 (11 cases), reflecting their structural similarity.

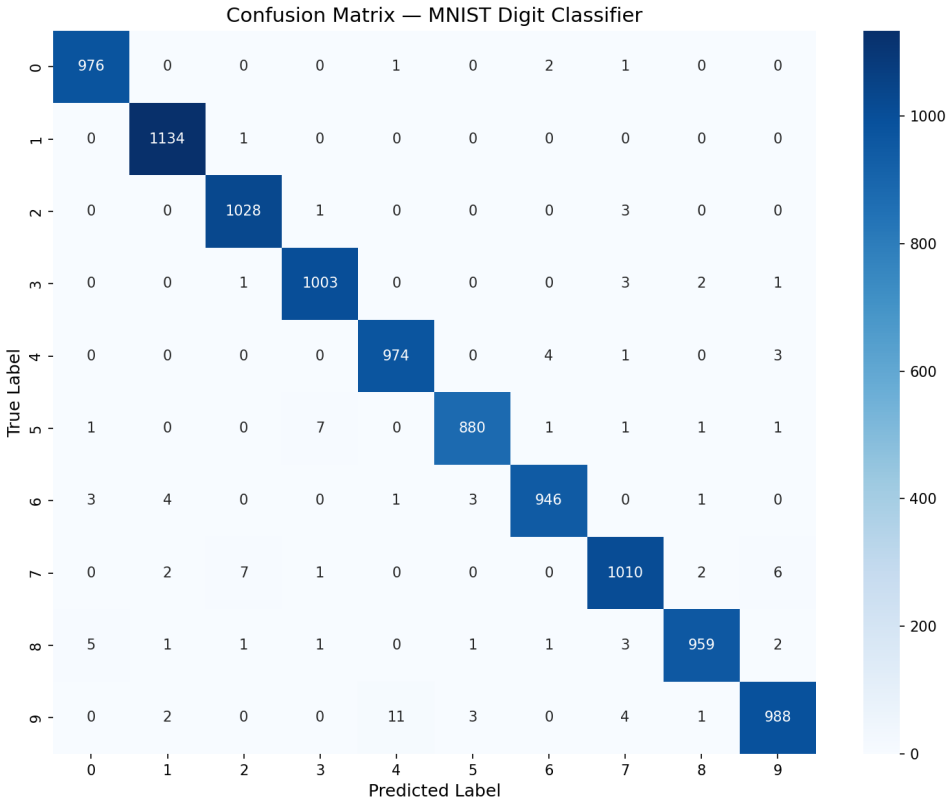



**Figure 7:** Confusion matrix on the MNIST test set (10,000 samples). Rows represent true labels and columns represent predicted labels. The strongly diagonal structure confirms high per-class accuracy. The most frequent misclassification is digit 9 predicted as 4 (11 cases).

**Understanding the Confusion Matrix.** While Precision, Recall, and F1 evaluate one digit class at a time, the **confusion matrix** shows all 10 classes simultaneously in a single $10 \times 10$ grid. Each row represents the *true* digit, and each column represents the *predicted* digit. Entry $C_{ij}$ counts how many images of true class $i$ were predicted as class $j$.

A **perfect classifier** would produce a purely diagonal matrix, every image of digit $i$ is predicted as $i$, so all off-diagonal entries are zero. Any off-diagonal entry $C_{ij}$ with $i \neq j$ represents a misclassification, the model confused digit $i$ for digit $j$.

To make this concrete, consider a tiny $3 \times 3$ example for digits 3, 4, and 9 only:

Reading this table: out of 1,009 real 9s in the test set, 988 were correctly identified, but **11 were mistakenly predicted as 4**. This is the single most common error in the entire model and is visible as the brightest off-diagonal cell in Figure 7. It is not surprising, the digits 4 and 9 share a similar upper structure and are frequently confused even by humans. The full $10 \times 10$ confusion matrix in Figure 7 extends this logic to all digit pairs simultaneously.

| True \ Pred | 3 | 4 | 9 |
|---|---|---|---|
| **3** | **1003** | 0 | 1 |
| **4** | 0 | **974** | 3 |
| **9** | 0 | **11** | **988** |

**Table 11:** Excerpt of the confusion matrix for digits 3, 4, and 9, taken from the actual results. The bold diagonal entries are correct predictions. The entry $C_{9,4} = 11$ reveals the most common misclassification in the entire model: digit 9 predicted as digit 4.

### Real-Time Inference

The application runs in real time on the laptop CPU described in Table 4. Once the model weights are loaded from `model.pth`, inference for a single 28×28 image completes in under 1 ms, making the Gradio interface highly responsive during live drawing. The confidence threshold of 0.6 defined in `config.yaml` triggers a low-confidence warning for ambiguous inputs, helping the user understand when the drawing may be unclear.

## Graphical Interface

The user interface is implemented using **Gradio** [6] and runs in any standard web browser at `http://localhost:7860`. It consists of:

- A **drawing canvas** (280×280 pixels, black brush on white background) where the user draws a digit with the mouse.
- A **label output** displaying the top predicted class and a bar chart showing confidence scores for all 10 digit classes.
- A **Clear** button to reset the canvas between drawings.
- A **Predict** button to trigger manual inference.

The canvas input is automatically preprocessed: converted to grayscale, inverted to match the MNIST format (white digit on black background), resized to 28×28 pixels using Lanczos resampling, and normalised before being passed to the model. The full preprocessing pipeline is described in Algorithm 2 and illustrated in Figure 9.

Figure 8 shows the application running in a web browser, where the user has drawn the digit 5 and the model correctly predicts it.

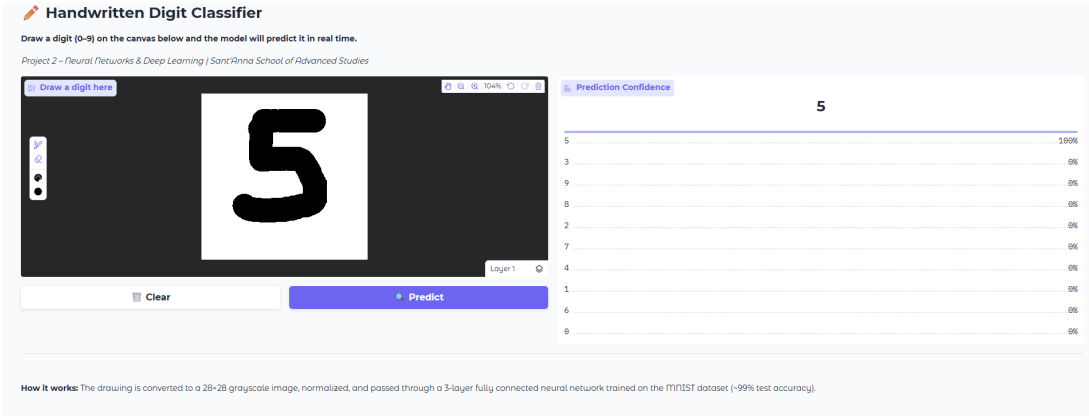



**Figure 8:** Screenshot of the Digit Classifier application running at `http://localhost:7860`. The user draws a digit on the left canvas using a black brush, and the model instantly displays the predicted class alongside a confidence bar chart for all 10 digit classes (0–9) on the right. The interface is built with Gradio 6.10.0 and runs in any standard web browser.

### Inference Pipeline

Algorithm 2 describes the real-time inference procedure executed each time the user draws on the canvas.

**Algorithm 2** Real-Time Digit Prediction

**Require:** Canvas image $I$ (RGBA, $280 \times 280$), trained weights $\theta^*$
**Ensure:** Predicted digit $\hat{c}$ and confidence scores $\hat{p}$

```
Convert I to grayscale → G (280 × 280)
Resize G to 28 × 28 using Lanczos resampling
Normalise: Ĝ ← (G − 0.1307) / 0.3081
Flatten: x ← vec(Ĝ) ∈ ℝ^784
Compute logits: z ← f_θ*(x) ∈ ℝ^10
Compute probabilities: p̂ ← Softmax(z)
ĉ ← arg max_c p̂_c
if max(p̂) < 0.6 then
    Display "Low confidence" warning
end if
return ĉ, p̂
```

**RGBA Input and Alpha Channel.** The image captured from the drawing canvas is represented in RGBA format, where the R, G, and B channels encode color information, and the A channel (alpha) represents pixel transparency (opacity). The alpha values typically range from 0 (fully transparent) to 255 (fully opaque).

In this application, the alpha channel is not used during preprocessing. This is because digit recognition depends only on the intensity and structural shape of the handwritten digit, which are fully captured by the RGB channels. The transparency information does not provide any meaningful contribution to the classification task and is therefore discarded.

Consequently, the preprocessing pipeline effectively reduces the RGBA image to a grayscale

representation using only the RGB components, ensuring consistency with the MNIST dataset, which consists of single-channel grayscale images.

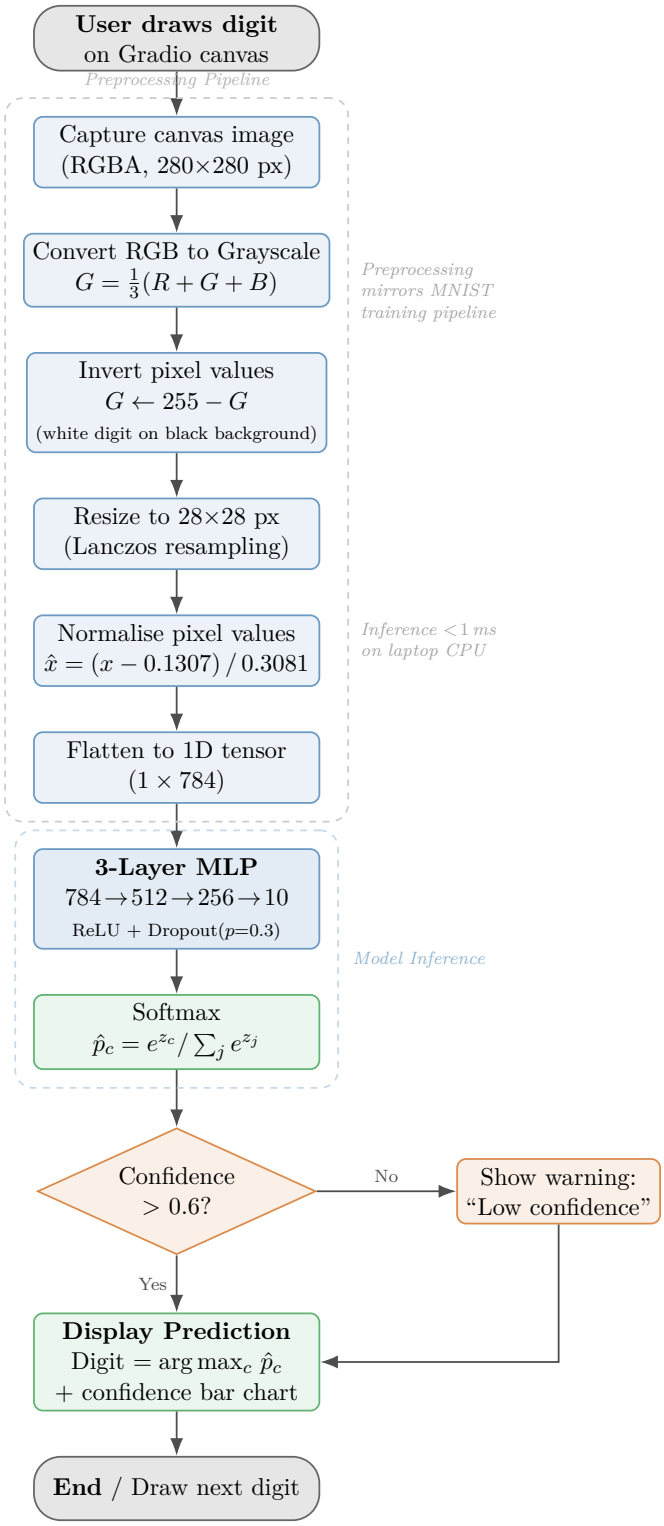



**Figure 9:** End-to-end pipeline of the Digit Classifier application. The canvas drawing is converted from RGB to grayscale, inverted to match the MNIST format (white digit on black background), resized to 28×28 pixels, normalised, and passed through the trained 3-layer MLP. The predicted digit is displayed with per-class confidence scores. If confidence falls below 0.6, a low-confidence warning is shown.

## Conclusion

This project demonstrates the design, training, and deployment of a 3-layer fully connected neural network for real-time handwritten digit classification. The network consists of 535,818 trainable parameters and was trained on the MNIST dataset for 50 epochs using the Adam optimiser ($\eta = 0.001$), Xavier uniform weight initialisation, Dropout regularisation ($p = 0.3$), and early stopping with patience $p = 7$ (which did not trigger, indicating stable convergence throughout).

The model achieves a **test accuracy of 98.98%** with a macro-averaged F1 score of 0.9897 across all 10 digit classes. The best validation loss recorded during training was 0.0626 at epoch 50, with a corresponding validation accuracy of 98.4%. The model runs interactively through a Gradio web interface at `http://localhost:7860`, where users can draw digits and observe predictions in real time with per-class confidence scores.

The modular MLOps-style code structure separates architecture, training, evaluation, and interface into independent files, each controlled by a central `config.yaml`. This design makes every hyperparameter traceable and reproducible without modifying source code. The project provides a hands-on demonstration of core deep learning concepts including feedforward networks, ReLU activations, Dropout regularisation, Cross-Entropy loss, Xavier initialisation, ReduceLROnPlateau scheduling, and Softmax classification.

## Xavier Initialisation: Worked Example

This appendix provides a concrete example of Xavier uniform initialisation as applied to the FC1 layer of the DigitClassifier, to illustrate how the weight matrix is populated before training begins.

The FC1 layer has fan_in = 784 and fan_out = 512, giving a bound of $a = 0.0680$. Every one of the $512 \times 784 = 401{,}920$ weights is independently sampled from $\mathcal{U}(-0.0680, +0.0680)$. A small corner of this matrix might look like:

$$W^{(1)} \approx \begin{pmatrix} 0.0523 & -0.0312 & 0.0187 & -0.0641 & \cdots \\ -0.0418 & 0.0055 & -0.0299 & 0.0612 & \cdots \\ 0.0034 & -0.0578 & 0.0401 & -0.0123 & \cdots \\ \vdots & \vdots & \vdots & \vdots & \ddots \end{pmatrix} \in \mathbb{R}^{512 \times 784}$$

To understand why this range matters, consider a simplified neuron in FC1 that receives only 3 inputs instead of 784. With fan_in = 3, fan_out = 2, the Xavier bound becomes $a = \sqrt{6/5} = 1.095$. Suppose the drawn weights and input pixel values are:

$$w_1 = 0.8, \quad w_2 = -0.6, \quad w_3 = 0.9 \qquad x_1 = 0.5, \quad x_2 = 0.3, \quad x_3 = 0.7$$

The pre-activation value is:

$$h = w_1 x_1 + w_2 x_2 + w_3 x_3 = 0.8 \times 0.5 - 0.6 \times 0.3 + 0.9 \times 0.7 = 0.4 - 0.18 + 0.63 = 0.85 \quad (33)$$

This is a healthy activation value, not too large to cause explosion, not too small to cause vanishing. Xavier initialisation ensures this property holds even when summing 784 terms

in the real FC1 layer, by scaling the weight range inversely with the square root of the total number of connections.

## Optimiser Comparison: Gradient Descent vs. Adam vs. SOAP

This appendix compares three optimisation algorithms: plain Gradient Descent (GD), Adam [2], and the recently proposed SOAP optimiser [7], to contextualise the choice of Adam in this project.

### Gradient Descent

The simplest optimiser. At each step, every weight is updated by moving in the direction opposite to the gradient, scaled by a fixed learning rate $\eta$:

$$\theta_t = \theta_{t-1} - \eta \cdot \mathbf{g}_t \tag{34}$$

Note that this is equivalent to a first-order iterative optimisation step: move the parameters in the direction of steepest descent of the loss surface, where the gradient $\mathbf{g}_t$ is computed via the chain rule of calculus, what the ML community calls *backpropagation*.

**Advantages:** simple, low memory (only stores $\theta$).
**Disadvantages:** the same $\eta$ is used for every weight regardless of gradient history. Requires careful manual tuning of $\eta$, too large causes divergence, too small causes slow convergence. Sensitive to noisy gradients.

### Adam

Adam [2] maintains two running averages per weight, momentum $\mathbf{m}_t$ and velocity $\mathbf{v}_t$, and uses them to compute an *adaptive* learning rate for each weight individually:

$$\theta_t = \theta_{t-1} - \eta \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \tag{35}$$

**Advantages:** adaptive per-weight learning rates; robust to noisy gradients; fast convergence in practice; works well out of the box with default hyperparameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$).
**Disadvantages:** higher memory cost than GD (stores $\mathbf{m}_t$ and $\mathbf{v}_t$ in addition to $\theta$); can sometimes converge to slightly suboptimal solutions compared to well-tuned SGD with momentum.

### SOAP

SOAP (Shampoo as Adam's Preconditioner) [7] is a very recent optimiser (2024) that combines the adaptivity of Adam with second-order curvature information. It maintains a preconditioner matrix for each layer and periodically updates it to account for the geometry of the loss surface:

$$\theta_t = \theta_{t-1} - \eta \cdot \mathbf{P}_t^{-1} \hat{\mathbf{m}}_t \tag{36}$$

where $\mathbf{P}_t$ is a layer-wise preconditioner matrix computed from the gradient history. This allows SOAP to take more informed steps that account for correlations between weights in the same layer.

**Advantages:** faster convergence than Adam in practice especially for large transformer models; better handling of ill-conditioned loss surfaces.
**Disadvantages:** significantly higher memory and computation cost (must store and invert preconditioner matrices); more complex to implement; primarily beneficial for very large models, overkill for a network of 535,818 parameters.

### Comparison Summary

| Property | Gradient Descent | Adam | SOAP |
|---|---|---|---|
| Update rule | $\eta \cdot g_t$ | $\eta \cdot \hat{m}/\sqrt{\hat{v}}$ | $\eta \cdot P^{-1}\hat{m}$ |
| Adaptive LR | No | Per weight | Per layer |
| Memory cost | Low | Medium | High |
| Curvature info | No | Partial | Yes |
| Ease of use | Simple | Simple | Complex |
| Best for | Simple tasks | Most deep learning | Large transformers |
| Year introduced | Classic | 2014 | 2024 |

**Table 12:** Comparison of three optimisers. Adam was selected for this project as it offers the best trade-off between performance, simplicity, and memory cost for a 535,818-parameter network.

**Why Adam was chosen for this project.** The DigitClassifier has 535,818 parameters, small enough that SOAP's additional complexity provides no practical benefit. Adam's adaptive learning rates make it robust to the varying gradient magnitudes across the three layers (FC1, FC2, FC3), and its default hyperparameters work well without manual tuning, which is ideal for a project of this scale.

## Epochs and Mini-Batches Explained

### The Core Problem

Training a neural network means finding the values of 535,818 weights that minimise the Cross-Entropy loss. To do this, we need to compute gradients from the training data and update the weights repeatedly. But how much data do we use at each update step, and how many times do we go through the dataset? This is where **epochs** and **mini-batches** come in.

### What is an Epoch?

An **epoch** is one complete pass through the *entire* training dataset. In this project, the training set has 54,000 images. After one epoch, every single image has been seen by the network exactly once. After 50 epochs, every image has been seen 50 times.

> *Think of it like studying for an exam. One epoch is reading your entire set of flashcards once from start to finish. Doing 50 epochs means going through all flashcards 50 times.*

### What is a Mini-Batch?

Rather than computing the gradient on all 54,000 images at once (which would be very slow and memory-intensive), or on just one image at a time (which would be very noisy), we split the training set into small groups called **mini-batches**. In this project, the batch size is $B = 64$.

At each training step, the network:

1. Takes 64 images from the training set
2. Runs a forward pass to compute predictions
3. Computes the loss over those 64 images
4. Runs backpropagation to compute gradients
5. Updates all 535,818 weights using Adam

This is repeated for every mini-batch in the dataset.

### How Many Steps Per Epoch?

With 54,000 training images and a batch size of 64:

$$\text{Steps per epoch} = \left\lceil \frac{54{,}000}{64} \right\rceil = 844 \text{ steps} \tag{37}$$

So in 50 epochs, Adam performs:

$$50 \times 844 = 42{,}200 \text{ weight update steps} \tag{38}$$

Each step updates all 535,818 weights, that is $42{,}200 \times 535{,}818 \approx 22.6$ *billion* individual weight adjustments during your training run.

### Summary Table

| **Concept** | **Definition** | **Value in this project** |
|---|---|---|
| Training set size | Total images for learning | 54,000 images |
| Batch size ($B$) | Images per update step | 64 images |
| Steps per epoch | $\lceil 54000/64 \rceil$ | 844 steps |
| Epochs | Full passes over training set | 50 epochs |
| Total update steps | Epochs $\times$ steps/epoch | 42,200 steps |
| Weights updated | Parameters per step | 535,818 |
| Total adjustments | Steps $\times$ weights | $\sim$22.6 billion |

**Table 13:** Epoch and mini-batch statistics for this project's training run.

### Visual Overview

Figure 10 illustrates the relationship between the dataset, mini-batches, and epochs.

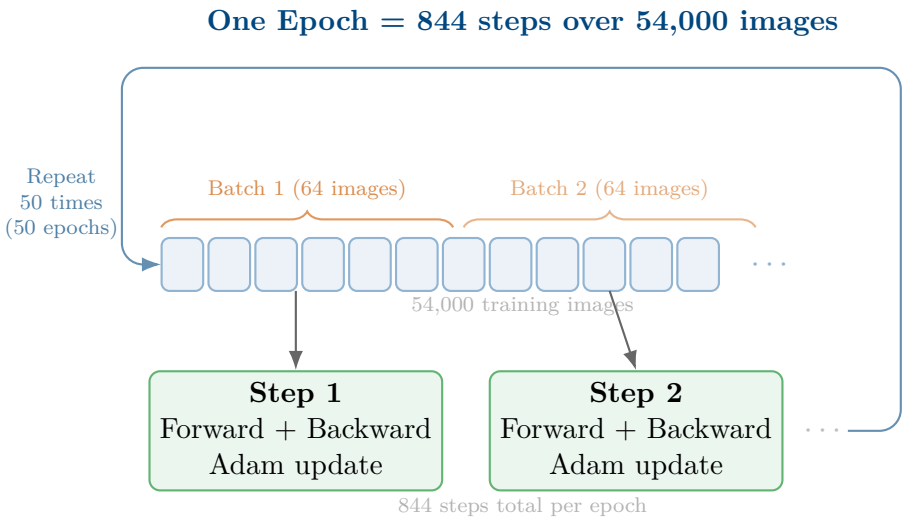



**Figure 10:** Relationship between the training dataset, mini-batches, update steps, and epochs. In each epoch, the 54,000 training images are divided into 844 mini-batches of 64 images each. For every mini-batch, one full forward pass, loss computation, backpropagation, and Adam weight update is performed. After 50 epochs, the network has completed 42,200 weight update steps in total.

### Why Not Use All 54,000 Images at Once?

Using all images at once is called **batch gradient descent**. It gives a very accurate gradient but requires loading all 54,000 images into GPU memory simultaneously, impractical for large datasets. More importantly, it finds a very smooth path downhill which often leads to sharp minima that generalise poorly to new data.

Using just 1 image at a time is called **stochastic gradient descent (SGD)**. It is fast and noisy, which can help escape poor local minima, but the gradient estimate is very inaccurate.

**Mini-batch** gradient descent (used here, $B = 64$) is the compromise, accurate enough gradients, small enough to fit in GPU memory, and noisy enough to generalise well.

| Method | Batch Size | Gradient Quality | Speed |
|---|---|---|---|
| Batch GD | All 54,000 | Very accurate | Very slow |
| Mini-batch GD | 64 (ours) | Good | Fast |
| Stochastic GD | 1 | Noisy | Very fast |

**Table 14:** Comparison of gradient descent variants by batch size. Mini-batch gradient descent with $B = 64$ was used in this project as it provides the best trade-off between gradient quality and computational speed.

## References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, *Gradient-based learning applied to document recognition*, Proceedings of the IEEE, 86(11):2278–2324, 1998.

[2] D. P. Kingma and J. Ba, *Adam: A Method for Stochastic Optimization*, arXiv:1412.6980, 2014.

[3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*, Journal of Machine Learning Research, 15:1929–1958, 2014.

[4] X. Glorot and Y. Bengio, *Understanding the difficulty of training deep feedforward neural networks*, Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS), pp. 249–256, 2010.

[5] V. Nair and G. E. Hinton, *Rectified Linear Units Improve Restricted Boltzmann Machines*, Proceedings of the 27th International Conference on Machine Learning (ICML), pp. 807–814, 2010.

[6] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, *Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild*, arXiv:1906.02569, 2019.

[7] N. Vyas, S. Morwani, R. Zhao, I. Shapira, D. Brandfonbrener, L. Janson, and S. Kakade, *SOAP: Improving and Stabilizing Shampoo using Adam*, arXiv:2409.11321, 2024.